GIẢI PHẪU CHI TIẾT THUẬT TOÁN PHASE 1 TRONG reference_solve.cpp

Vét cạn Lộ trình Nhánh-Cận, Cắt tỉa Thống trị, Thư giãn Đối ngẫu Lagrangian và Luyện kim Mô phỏng Đẩy-Lấp trên Không gian Đóng gói Tập hợp (Set Packing)

Tài liệu Kỹ thuật & Chứng minh Toán học Chi tiết — Dự án Tối ưu hóa Lộ trình Ghép xe Khách & Hàng (SARP)

Mục lục

1 Tổng quan Tư tưởng và Triết lý Thiết kế Phase 1 **2**
- 1.1 Bản chất Hình học của Bài toán SARP với Khung thời gian Hẹp . . . 2
- 1.2 Sơ đồ Chuỗi 5 Bài toán con trong Phase 1 (column_solve) . . . 2

2 Bài toán con 0: Đồng nhất hóa Đồ thị Tác vụ (Macro-Node Contraction) **2**
- 2.1 Động cơ Toán học của Phép Co Đỉnh (Node Contraction) . . . 2
- 2.2 Công thức Ánh xạ Tường minh (Lines 574–604) . . . 3

3 Bài toán con 1: Vét cạn Lộ trình Nhánh-Cận theo từng Xe (enum_dfs) **4**
- 3.1 Phát biểu Toán học của Bài toán Sinh Cột Đơn Xe . . . 4
- 3.2 Cấu trúc Trạng thái Đệ quy trong enum_dfs (Lines 378–407) . . . 5
- 3.3 Hệ thống 4 Định lý Cắt tỉa Chính xác (Exact Branch-and-Bound Pruning) . . . 5
  - 3.3.1 Nhát cắt 1: Cắt tỉa Tìm kiếm Nhị phân theo Hạn chót Đón (lower_bound) . 6
  - 3.3.2 Nhát cắt 2: Cắt tỉa Tải trọng Tức thời (Capacity Check) . . . 6
  - 3.3.3 Nhát cắt 3: Cắt tỉa Khả năng Giao thẳng của Kiện hàng Mới (Self Reachability) 6
  - 3.3.4 Nhát cắt 4: Cắt tỉa Nhìn trước Tập Hàng đang Mở (open_ok Lookahead) . . 6

4 Bài toán con 2: Cắt tỉa Thống trị Đẳng cấu Tập Yêu cầu (enum_record) **7**
- 4.1 Định lý Thống trị Cột trên Không gian Set Packing . . . 7
- 4.2 Cơ chế Băm 64-bit và Tổ chức Bộ nhớ Phẳng (Lines 345–372) . . . 8

5 Bài toán Chủ (Master Problem) và Bài toán con 3: Khởi tạo Tham lam **8**
- 5.1 Mô hình Quy hoạch Nguyên Đóng gói Tập hợp (Set Packing Master Problem) . . . 8
- 5.2 Bài toán con 3: Khởi tạo Tham lam theo Lợi nhuận (Greedy Set Packing, Lines 432–459) . . . 9

6 Bài toán con 4: Thư giãn Đối ngẫu Lagrangian và Tối ưu hóa Dưới đạo hàm **10**
- 6.1 Xây dựng Bài toán Thư giãn Lagrangian $LR(u)$ . . . 10
- 6.2 Phân rã thành K Bài toán con Độc lập Tuyệt đối (Lines 472–481) . . . 10
- 6.3 Các Định lý Toán học về Cận trên Đối ngẫu và Độ lệch bù . . . 11
- 6.4 Thuật toán Tối ưu hóa Dưới đạo hàm với Bước nhảy Polyak (Lines 465–496) . . . 12

7 Bài toán con 5: Luyện kim Mô phỏng trên Không gian Cột với Toán tử Đẩy-và-Lấp (Eject-and-Refill SA) **13**
- 7.1 Bản chất của Toán tử Láng giềng Đẩy-và-Lấp (Eject-and-Refill Operator) . . . 13
- 7.2 Giải phẫu Từng Bước trong 1 Vòng lặp Set Packing SA (Lines 505–550) . . . 13

7.2.1 Bước 1: Chuẩn bị Danh sách Cột theo Thứ tự Lợi nhuận (`tcols[k]`) 13
7.2.2 Bước 2: Lấy mẫu Cột Ứng viên Đa tầng (Biased Candidate Sampling, Lines 522–526) 13
7.2.3 Bước 3: Thực thi Chuỗi Đẩy-và-Lấp có Ghi Nhật ký Hoàn tác (Lines 527–538) 13
7.2.4 Bước 4: Tiêu chuẩn Chấp nhận Metropolis, Làm nguội Hàm mũ và Hoàn tác $O(1)$ 14

8 Thực nghiệm Toàn diện và Chứng minh Tối ưu Toàn cục trên 8 Bộ Test 15
8.1 Phương pháp Kiểm chứng Độc lập bằng Quy hoạch Tuyến tính (`GLOP`) và Quy hoạch Nguyên (`SCIP`) 15
8.2 Bảng Tổng hợp Kết quả Thực nghiệm trên Toàn bộ 8 Testcase 15
8.3 Phân tích Chuyên sâu Kết quả Thực nghiệm 16

9 Xác định Chặt chẽ các Ràng buộc và Điều kiện để Phase 1 Đảm bảo Tính Chính xác Tuyệt đối 16
9.1 Điều kiện 1: Bất đẳng thức Tam giác Thời gian Mạnh (Time Triangle Inequality) . . 16
9.2 Điều kiện 2: Tính Đơn điệu Không âm của Thời gian Di chuyển và Phục vụ 17
9.3 Điều kiện 3: Cấu trúc Khung Thời gian của Khách hàng trong Phép Co Macro-Node 17
9.4 Điều kiện 4: Tính Độc lập Hoàn toàn Giữa các Xe cho Định lý Thống trị Cột (`enum_record`) 17
9.5 Điều kiện 5: Bảng Băm Không Xử lý Đụng độ (Un-probed Hash Table) tại Line 363 18
9.6 Điều kiện 6: Điều kiện Cần và Đủ để Kết luận Nghiệm Cuối Phase 1 là Tối ưu Toàn cục 18

1 Tổng quan Tư tưởng và Triết lý Thiết kế Phase 1

1.1 Bản chất Hình học của Bài toán SARP với Khung thời gian Hẹp

Bài toán **Share-a-Ride Problem (SARP)** yêu cầu điều phối một đội gồm K xe taxi xuất phát từ các bến đỗ O_k với tải trọng tối đa Q_k để phục vụ hai nhóm yêu cầu:

- N **yêu cầu chở khách (Passengers)**: Đón tại P_i trong khung thời gian $[e_i, l_i]$ và phải chở **trực tiếp ngay lập tức** tới điểm trả D_i (không được phép ghé bất kỳ điểm dừng trung gian nào khác khi khách đang ngồi trên xe).
- M **yêu cầu chở hàng (Parcels)**: Đón gói hàng nặng W_j tại P_j trong khung thời gian $[e_j^p, l_j^p]$ và trả tại D_j trong khung thời gian $[e_j^d, l_j^d]$, cho phép ghép chung nhiều gói hàng hoặc chở xen kẽ khách hàng giữa lúc đón và trả hàng nếu không vượt tải trọng Q_k.

Mục tiêu của bài toán là **tối đa hóa tổng lợi nhuận ròng (Total Net Benefit)**: tổng doanh thu Rev_r của các yêu cầu được phục vụ trừ đi tổng chi phí di chuyển $C(u, v)$ trên toàn đội xe.

Nhận xét Cốt lõi Quyết định Kiến trúc Thuật toán

Trong các bộ dữ liệu thực tế của bài toán SARP (từ `test_10.txt` đến `test_500.txt`), khung thời gian $[e_r, l_r]$ rất chặt so với thời gian di chuyển $T(u, v)$ và thời gian phục vụ S_r. Hệ quả là **mỗi xe chỉ có thể phục vụ tối đa từ 2 đến 5 yêu cầu trong suốt hành trình**.

- **Điểm yếu của Local Search / ALNS / Tabu Search trên lộ trình**: Khi lộ trình chỉ dài $3 \sim 4$ yêu cầu, mỗi yêu cầu chiếm tới $25\% \sim 33\%$ lợi nhuận của xe. Một phép nhiễu (Ruin) tháo 15% số yêu cầu sẽ dễ dàng phá hỏng các tổ hợp ghép chéo duy nhất, tạo ra dao động phương sai $\approx 2.5\% \sim 3.0\%$ quanh đỉnh tối ưu.
- **Điểm yếu của MILP trực tiếp theo cạnh (Arc-Based MILP)**: Ràng buộc nối thời gian qua hằng số Big-M ($t_v \geq t_u + S_u + T_{u,v} - M(1 - x_{k,u,v})$) làm nghiệm thư giãn tuyến tính (LP Relaxation) bị suy biến nặng, khiến cây Branch-and-Bound bùng nổ.
- **Ưu thế tuyệt đối của Phân rã Dantzig-Wolfe (Phase 1)**: Vì mỗi xe chỉ ghép được một số lượng rất nhỏ yêu cầu, tổng số lộ trình hợp lệ của một xe chỉ vào khoảng vài trăm đến vài nghìn. Ta có thể **vét cạn chính xác** 100% **mọi lộ trình hợp lệ của từng xe** trong vài mili-giây, biến bài toán định tuyến phức tạp thành bài toán **Đóng gói Tập hợp có Trọng số (Maximum Weight Set Packing)** thuần túy!

1.2 Sơ đồ Chuỗi 5 Bài toán con trong Phase 1 (`column_solve`)

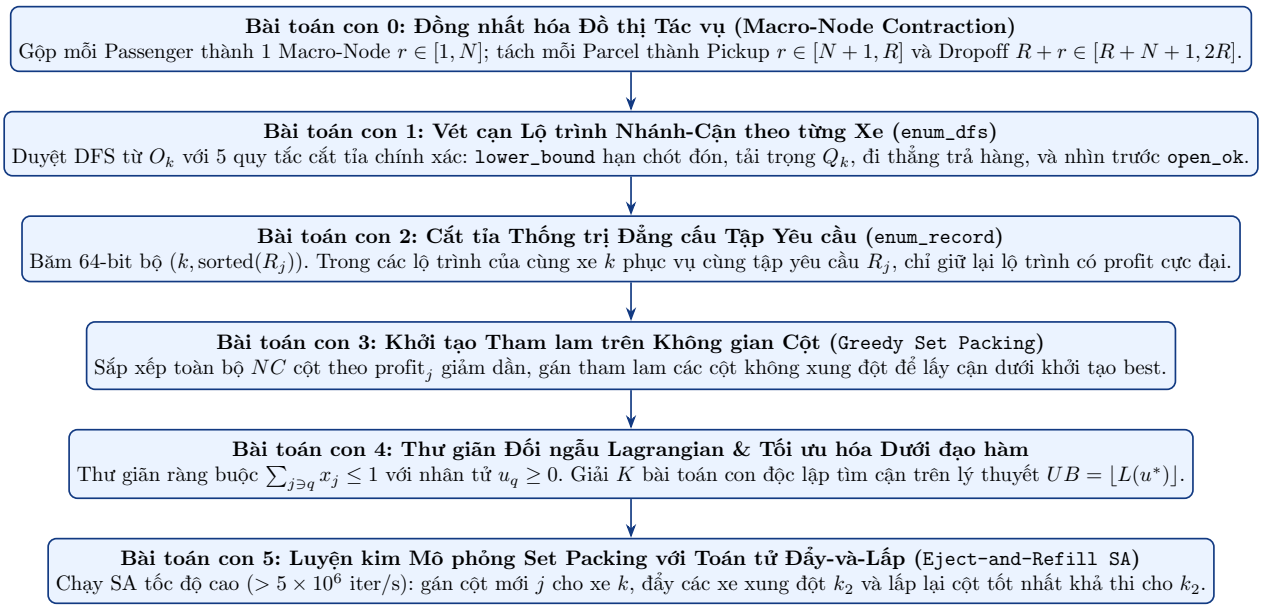
Thuật toán Phase 1 trong `reference_solve.cpp` phân rã bài toán SARP thành một chuỗi **5 bài toán con nối tiếp nhau** (Hình 1):

—

2 Bài toán con 0: Đồng nhất hóa Đồ thị Tác vụ (Macro-Node Contraction)

2.1 Động cơ Toán học của Phép Co Đỉnh (Node Contraction)

Trong phát biểu gốc, cả khách hàng $i \in [1, N]$ lẫn kiện hàng $j \in [N+1, N+M]$ đều có hai đỉnh vật lý là điểm đón P và điểm trả D. Tuy nhiên, ràng buộc nghiệp vụ của hai loại yêu cầu này hoàn toàn khác nhau:



Hình 1: Kiến trúc 5 bài toán con nối tiếp trong Phase 1 (`column_solve`) của `reference_solve.cpp`.

1. Với **khách hàng** $r \in [1, N]$: Sau khi xe đến đỉnh đón P_r tại thời điểm $t_{\text{arr}} \le l_r$, xe bắt đầu phục vụ tại $\max(t_{\text{arr}}, e_r)$, mất S_r đơn vị thời gian để khách lên xe, đi thẳng tới D_r mất $T(P_r, D_r)$ với chi phí $C(P_r, D_r)$, và mất thêm S_r đơn vị thời gian để khách xuống xe tại D_r. Trong suốt quá trình từ P_r đến D_r, xe **tuyệt đối không được rẽ đi đâu khác**.
2. Với **kiện hàng** $r \in [N+1, R]$ $(R = N + M)$: Giữa sự kiện đón hàng tại P_r và trả hàng tại D_r, xe có thể ghé thăm tùy ý các điểm đón/trả của khách hoặc của kiện hàng khác miễn là tổng tải trọng không vượt Q_k và kịp trả hàng trong $[e_r^d, l_r^d]$.

Bài toán con 2.1 (Biến đổi Đồ thị Vật lý sang Đồ thị Tác vụ $2R$ Đỉnh)**.** Biểu diễn mọi hành động đón/trả khách và hàng về một không gian chỉ số tác vụ thống nhất $x \in \{1, \ldots, 2R\}$ sao cho mỗi chỉ số x đóng vai trò một "nguyên tử định tuyến" (routing atom) có điểm vào, điểm ra, thời lượng nội bộ, chi phí nội bộ và biến thiên tải trọng xác định.

2.2 Công thức Ánh xạ Tường minh (Lines 574–604)

Thuật toán định nghĩa $2R$ node tác vụ thông qua 6 mảng thuộc tính $\text{nin}[x], \text{nout}[x], \text{ndur}[x], \text{nic}[x], \text{nw}[x]$ và $[ne[x], nl[x]]$ như trong Bảng 1:

Mảng C++	Ý nghĩa Vật lý / Toán học	**Khách hàng** $x = r \in [1, N]$	**Đón hàng (Pickup)** $x = r \in [N+1, R]$	**Trả hàng (Dropoff)** $x = R + r \in [R+N+1, 2R]$
`nin[x]`	Đỉnh đồ thị vật lý khi xe đi vào node x	P_r	P_r	D_r
`nout[x]`	Đỉnh đồ thị vật lý khi xe rời khỏi node x	D_r	P_r	D_r
`ndur[x]`	Tổng thời gian thực hiện bên trong node x	$2S_r + T(P_r, D_r)$	S_r	S_r
`nic[x]`	Chi phí di chuyển nội bộ bên trong node x	$C(P_r, D_r)$	0	0
`nw[x]`	Biến thiên tải trọng hàng hóa khi qua x	0	$+W_r$	$-W_r$
`ne[x]`	Thời điểm sớm nhất bắt đầu tại `nin[x]`	e_r	e_r^p	e_r^d
`nl[x]`	Thời điểm muộn nhất phải đến `nin[x]`	l_r	l_r^p	l_r^d

Bảng 1: Bảng ánh xạ đồng nhất hóa Passenger (1 Macro-Node) và Parcel (2 Split-Nodes).

Mệnh đề 2.1 (Tính bất biến của Phương trình Chuyển trạng thái)**.** *Với cách định nghĩa trên, khi xe đang ở đỉnh vật lý last tại thời điểm time và quyết định đi tới bất kỳ node tác vụ* $x \in [1, 2R]$ *(dù*

*x là khách hàng, điểm đón hàng hay điểm trả hàng), phương trình cập nhật thời gian đến (arr) và thời điểm hoàn tất rời khỏi nout[x] (dep) có dạng **hoàn toàn giống nhau**:*

$$arr(x) = time + T(last, nin[x]) \tag{1}$$
$$Feasibility(x) \iff arr(x) \le nl[x] \tag{2}$$
$$dep(x) = \max\big(arr(x), ne[x]\big) + ndur[x] \tag{3}$$

Chứng minh. Kiểm tra trực tiếp cho trường hợp khách hàng $x = r \in [1, N]$: Xe đến $P_r = \text{nin}[r]$ lúc $\text{arr} = \text{time} + T(\text{last}, P_r) \le l_r$. Xe bắt đầu đón khách lúc $t_{\text{start}} = \max(\text{arr}, e_r)$, rời P_r lúc $t_{\text{start}} + S_r$, đến D_r lúc $t_{\text{start}} + S_r + T(P_r, D_r)$ và hoàn tất trả khách tại $D_r = \text{nout}[r]$ đúng vào lúc:

$$t_{\text{finish}} = \max(\text{arr}, e_r) + \underbrace{2S_r + T(P_r, D_r)}_{\text{ndur}[r]} = \text{dep}(r).$$

Chi phí phát sinh trên chặng này gồm chi phí đi từ last tới P_r cộng chi phí đi từ P_r tới D_r, tức là $C(\text{last}, \text{nin}[r]) + \text{nic}[r]$. Do đó phép co Macro-Node bảo toàn chính xác 100% thời gian và chi phí! □

Ngoài ra, bước tiền xử lý thực hiện lọc khả thi tĩnh (`servable[r]`):

$$\text{servable}[r] = \begin{cases} 0, & \text{nếu } (r \le N \wedge e_r > l_r) \vee (r > N \wedge (e_r^p > l_r^p \vee e_r^d > l_r^d \vee W_r > \max_k Q_k)), \\ 1, & \text{ngược lại.} \end{cases}$$

Mọi yêu cầu có $\text{servable}[r] = 1$ được đưa vào danh sách `servable_list`.

—

3 Bài toán con 1: Vét cạn Lộ trình Nhánh-Cận theo từng Xe (`enum_dfs`)

3.1 Phát biểu Toán học của Bài toán Sinh Cột Đơn Xe

Bài toán con 3.1 (Single-Vehicle Exact Route Enumeration)**.** Cho xe $k \in \{1, \dots, K\}$ xuất phát từ đỉnh O_k tại thời điểm $t_0 = 0$ với tải trọng ban đầu $\text{load}_0 = 0$. Hãy liệt kê toàn bộ các chuỗi tác vụ hợp lệ $\sigma = (x_1, x_2, \dots, x_m)$ $(m \ge 1, x_s \in [1, 2R])$ thỏa mãn đồng thời 5 điều kiện:

(C1) **Tính đơn nhất (Simple Requests)**: Mỗi yêu cầu $q \in \mathcal{R}$ được đón tối đa 1 lần trong σ.

(C2) **Tính đóng của Hàng hóa (Parcel Pairing & Precedence)**: Mọi kiện hàng $q \in [N+1, R]$ xuất hiện trong σ đều phải có node trả $R + q$ nằm sau node đón q trong σ. Khi kết thúc lộ trình σ, trên xe không còn kiện hàng nào chưa trả ($\text{e_open} = \emptyset$).

(C3) **Ràng buộc Khung thời gian (Time Windows)**: Tại mọi bước $s = 1, \dots, m$, thời điểm đến t_s^{arr} và rời t_s^{dep} xác định đệ quy theo (1)–(3) (với $\text{nout}[x_0] \equiv O_k, t_0^{\text{dep}} = 0$) phải thỏa mãn $t_s^{\text{arr}} \le nl[x_s]$.

(C4) **Ràng buộc Tải trọng (Capacity)**: Tại mọi bước $s = 1, \dots, m$, tải trọng tích lũy $\text{load}_s = \sum_{i=1}^{s} \text{nw}[x_i] \le Q_k$.

(C5) **Tính Sinh lời Ròng (Positive Net Profit)**: Lợi nhuận ròng của lộ trình phải dương nghiêm ngặt:

$$\text{profit}(\sigma) = \sum_{q \in \mathcal{S}(\sigma)} \text{Rev}_q - \sum_{s=1}^{m} \Big(C(\text{nout}[x_{s-1}], \text{nin}[x_s]) + \text{nic}[x_s] \Big) > 0, \tag{4}$$

trong đó $\mathcal{S}(\sigma) = \{x_s \in \sigma : x_s \le R\}$ là tập các yêu cầu được phục vụ bởi σ.

3.2 Cấu trúc Trạng thái Đệ quy trong enum_dfs (Lines 378–407)

Hàm đệ quy được khai báo với chữ ký:

```
static void enum_dfs(int k, int last, ll time, int load, ll profit)
```

Trong đó:

- `k`: Chỉ số xe hiện tại ($1 \le k \le K$).
- `last`: Đỉnh vật lý nơi xe vừa hoàn tất tác vụ gần nhất (khởi tạo `last` $= O_k$).
- `time`: Thời điểm sớm nhất xe sẵn sàng rời khỏi `last` (khởi tạo `time` $= 0$).
- `load`: Tổng khối lượng hàng hóa đang nằm trên xe (khởi tạo `load` $= 0$).
- `profit`: Tổng doanh thu đã ghi nhận trừ đi tổng chi phí di chuyển tính đến thời điểm hiện tại (khởi tạo `profit` $= 0$).

Đi kèm là 4 cấu trúc dữ liệu trạng thái toàn cục được cập nhật theo cơ chế *backtracking* ($O(1)$ push/pop):

- `e_seq`: Vector lưu chuỗi các node tác vụ $(x_1, x_2, \dots)$ đã đi qua từ đầu lộ trình.
- `e_reqs`: Vector lưu tập các mã yêu cầu $q \in [1, R]$ đã được xe đón.
- `e_open`: Vector lưu danh sách các kiện hàng $q \in [N+1, R]$ **đã đón lên xe nhưng chưa trả**.
- `e_used[q]`: Mảng đánh dấu nhị phân $\in \{0, 1\}$ cho biết yêu cầu q đã xuất hiện trong lộ trình hiện tại hay chưa.

Lưu ý Quan trọng về Dấu của profit tại các Trạng thái Trung gian

Trong `enum_dfs`:

- Khi xe **đón một khách hàng** $q \le N$ (Line 403): vì khách được chở thẳng từ P_q tới D_q trong cùng Macro-Node q, doanh thu được cộng ngay lập tức:

$$\text{profit}_{\text{new}} = \text{profit} - C(\text{last}, \text{nin}[q]) - \text{nic}[q] + \text{Rev}[q].$$

- Khi xe **đón một kiện hàng** $q > N$ (Line 404): hàng mới chỉ lên xe chứ chưa giao xong, nên thuật toán **chỉ trừ chi phí đi đón hàng** mà chưa cộng doanh thu:

$$\text{profit}_{\text{new}} = \text{profit} - C(\text{last}, \text{nin}[q]).$$

- Khi xe **trả kiện hàng đó** tại $x = R + q$ (Line 390): lúc này doanh thu $\text{Rev}[q]$ mới được cộng vào:

$$\text{profit}_{\text{new}} = \text{profit} - C(\text{last}, \text{nin}[R+q]) + \text{Rev}[q].$$

Do đó, tại các bước trung gian (hoặc ngay cả khi vừa phục vụ một yêu cầu đầu tiên ở xa depot làm chi phí rỗng đầu tuyến lớn), profit có thể **nhận giá trị âm**, rồi sau khi ghép thêm các yêu cầu tiếp theo mới trở thành dương! Vì vậy, **`enum_dfs` tuyệt đối không cắt nhánh khi profit** ≤ 0, mà chỉ kiểm tra điều kiện profit > 0 khi ghi nhận một cột hoàn chỉnh tại Line 382:

```
if (e_open.empty() && !e_seq.empty() && profit > 0) enum_record(k, profit);
```

3.3 Hệ thống 4 Định lý Cắt tỉa Chính xác (Exact Branch-and-Bound Pruning)

Sức mạnh giúp `enum_dfs` duyệt toàn bộ đề bài 500 yêu cầu chỉ trong 65,265 trạng thái (9.5 ms) nằm ở 4 nhát cắt bảo toàn tính tối ưu sau đây:

3.3.1 Nhát cắt 1: Cắt tỉa Tìm kiếm Nhị phân theo Hạn chót Đón (lower_bound)

Trước khi bắt đầu gọi enum_dfs, thuật toán sắp xếp toàn bộ các yêu cầu khả thi e_by_l theo thứ tự không giảm của hạn chót đón $nl[q]$ (Lines 411–413):

$$nl[\text{e_by_l}[0]] \le nl[\text{e_by_l}[1]] \le \cdots \le nl[\text{e_by_l}[|\mathcal{R}|-1]],$$

và lưu mảng giá trị tương ứng vào $\texttt{e_l}[i] = nl[\text{e_by_l}[i]]$.

Bổ đề 3.1 (Cắt tỉa Tiền tố Quá hạn bằng Tìm kiếm Nhị phân)**.** *Giả sử thời gian di chuyển giữa mọi cặp đỉnh là không âm: $T(u,v) \ge 0, \forall u,v$. Tại trạng thái đệ quy có thời điểm hiện tại là time, mọi yêu cầu q có $nl[q] <$ time đều không thể đón kịp.*

Chứng minh. Thời điểm xe đến điểm đón $\text{nin}[q]$ là $\text{arr} = \text{time} + T(\text{last}, \text{nin}[q])$. Vì $T(\text{last}, \text{nin}[q]) \ge 0$ nên $\text{arr} \ge \text{time} > nl[q]$, vi phạm điều kiện (2). □

Nhờ Bổ đề này, tại Line 394, thay vì duyệt toàn bộ R yêu cầu từ 0 đến $R-1$, vòng lặp mở rộng yêu cầu mới nhảy thẳng tới vị trí đầu tiên có $nl[q] \ge$ time bằng tìm kiếm nhị phân trong $O(\log R)$:

```
for (size_t idx = lower_bound(e_l.begin(), e_l.end(), time) - e_l.begin(); idx <
    e_by_l.size(); ++idx)
```

Khi time tăng dần dọc theo độ sâu của lộ trình, chỉ số idx tiến nhanh về phía cuối mảng e_by_l, làm hệ số phân nhánh (branching factor) giảm theo cấp số nhân!

3.3.2 Nhát cắt 2: Cắt tỉa Tải trọng Tức thời (Capacity Check)

Tại Line 400, khi thử đón một kiện hàng $q > N$:

$$\text{load} + \text{nw}[q] > Q_k \implies \text{continue (cắt nhánh)}.$$

Vì khách hàng $q \le N$ có $\text{nw}[q] = 0$, phép kiểm tra này chỉ cần thực hiện khi $q > N$.

3.3.3 Nhát cắt 3: Cắt tỉa Khả năng Giao thẳng của Kiện hàng Mới (Self Reachability)

Khi thử đón một kiện hàng $q > N$, giả sử xe đến kịp điểm đón ($\text{arr} \le nl[q]$) và hoàn tất bốc hàng tại thời điểm $\text{dep} = \max(\text{arr}, ne[q]) + \text{ndur}[q]$. Để kiện hàng q có thể được trả hợp lệ tại $R+q$ ở một bước nào đó trong tương lai, ít nhất nếu xe đi thẳng ngay lập tức từ điểm đón $\text{nout}[q] = P_q$ tới điểm trả $\text{nin}[R+q] = D_q$, xe phải đến trước hạn chót trả hàng $nl[R+q]$:

$$\text{dep} + T(\text{nout}[q], \text{nin}[R+q]) > nl[R+q] \implies \text{continue (cắt nhánh - Line 400)}. \tag{5}$$

3.3.4 Nhát cắt 4: Cắt tỉa Nhìn trước Tập Hàng đang Mở (open_ok Lookahead)

Đây là nhát cắt tinh tế nhất của enum_dfs (Lines 374–377):

```
// every still-open parcel (except 'skip') must remain deliverable directly from (v
    , dep)
static inline bool open_ok(ll dep, int v, int skip) {
    for (int o : e_open) if (o != skip && dep + T(v, nin[R + o]) > nl[R + o])
        return false;
    return true;
}
```

Mệnh đề 3.2 (Tính đúng đắn của Cắt tỉa open_ok)**.** *Giả sử ma trận thời gian T thỏa mãn bất đẳng thức tam giác ($T(v,w) + T(w,u) \ge T(v,u)$). Khi xe chuẩn bị thực hiện một bước chuyển tới đỉnh tác vụ tiếp theo (kết thúc tại đỉnh vật lý v vào thời điểm dep), nếu tồn tại bất kỳ kiện hàng nào $o \in e_open$ đang nằm trên xe mà:*

$$dep + T(v, nin[R+o]) > nl[R+o], \tag{6}$$

thì không tồn tại bất kỳ cách mở rộng lộ trình hợp lệ nào có thể trả được kiện hàng o đúng hạn.

Chứng minh. Trong mọi lộ trình hoàn chỉnh mở rộng từ trạng thái (v, dep), kiện hàng $o \in \text{e_open}$ bắt buộc phải được trả tại đỉnh $\text{nin}[R+o]$ ở một bước $s \geq 1$ nào đó sau khi rời v. Do thời gian phục vụ $\text{ndur} \geq 0$, thời gian chờ ≥ 0 và bất đẳng thức tam giác của T, thời điểm đến $\text{nin}[R+o]$ trong mọi lộ trình mở rộng luôn thỏa mãn:

$$t^{\text{arr}}(R+o) \geq \text{dep} + T(v, \text{nin}[R+o]) > nl[R+o],$$

tức kiện hàng o chắc chắn bị trễ hạn trả. Do đó cắt bỏ nhánh này bảo toàn 100% tập lộ trình hợp lệ. □

Hàm `open_ok` được gọi tại cả 2 nhánh:

- Khi thử trả kiện hàng $q \in \text{e_open}$ (Line 388): Gọi `open_ok(dep, nout[R+q], q)` để đảm bảo việc đi trả q không làm các kiện hàng khác $o \neq q$ đang trên xe bị quá hạn trả.
- Khi thử đón yêu cầu mới q (Line 401): Gọi `open_ok(dep, nout[q], -1)` để đảm bảo việc rẽ đi đón q không làm bất kỳ kiện hàng nào đang nằm trên xe bị quá hạn trả.

—

4 Bài toán con 2: Cắt tỉa Thống trị Đẳng cấu Tập Yêu cầu (`enum_record`)

4.1 Định lý Thống trị Cột trên Không gian Set Packing

Một tập gồm m yêu cầu trên cùng một xe k có thể có nhiều thứ tự đón/trả hợp lệ khác nhau (ví dụ: đón hàng $A \to$ đón hàng $B \to$ trả $A \to$ trả B, hoặc đón $A \to$ đón $B \to$ trả $B \to$ trả A, hoặc đón khách $C \to$ đón hàng $A \to$ trả hàng A). Nếu giữ lại tất cả các hoán vị này, số lượng cột trong bài toán Set Packing sẽ bị phình to không cần thiết.

Định lý 4.1 (Định lý Thống trị Đẳng cấu Tập Yêu cầu — Column Dominance)**.** *Xét hai lộ trình hoàn chỉnh hợp lệ σ_1 và σ_2 của cùng một xe $k \in \{1, \ldots, K\}$. Giả sử σ_1 và σ_2 phục vụ đúng cùng một tập hợp các yêu cầu:*

$$\mathcal{S}(\sigma_1) = \mathcal{S}(\sigma_2) = R_j \subseteq \{1, \ldots, R\},$$

*và $\text{profit}(\sigma_1) \geq \text{profit}(\sigma_2)$. Khi đó, tồn tại nghiệm tối ưu toàn cục của bài toán SARP không sử dụng lộ trình σ_2 (nói cách khác, σ_1 **thống trị hoàn toàn** σ_2).*

Chứng minh. Giả sử tồn tại một phương án điều phối toàn đội xe $\Pi = (\pi_1, \ldots, \pi_K)$ hợp lệ sử dụng lộ trình $\pi_k = \sigma_2$ trên xe k. Vì tính hợp lệ giữa các xe khác nhau chỉ phụ thuộc vào việc các tập yêu cầu $\mathcal{S}(\pi_1), \ldots, \mathcal{S}(\pi_K)$ đôi một rời nhau ($\mathcal{S}(\pi_u) \cap \mathcal{S}(\pi_v) = \emptyset, \forall u \neq v$), khi ta thay thế $\pi_k = \sigma_2$ bằng $\pi'_k = \sigma_1$:

1. Lộ trình σ_1 đã hợp lệ nội bộ trên xe k (được chứng nhận bởi `enum_dfs`).
2. Tập yêu cầu tiêu thụ trên xe k không đổi: $\mathcal{S}(\sigma_1) = \mathcal{S}(\sigma_2) = R_j$, nên không gây ra bất kỳ xung đột mới nào với các xe $u \neq k$.
3. Tổng lợi nhuận toàn đội xe thay đổi một lượng $\Delta = \text{profit}(\sigma_1) - \text{profit}(\sigma_2) \geq 0$.

Do đó ta có thể loại bỏ hoàn toàn σ_2 ngay tại thời điểm sinh cột mà không làm suy giảm giá trị tối ưu toàn cục! □

4.2 Cơ chế Băm 64-bit và Tổ chức Bộ nhớ Phẳng (Lines 345–372)

Để thực thi Định lý trên trong thời gian trung bình $O(|R_j| \log |R_j|)$ cho mỗi lộ trình tìm được, hàm `enum_record(int k, ll profit)` thực hiện:

1. **Chuẩn hóa Tập yêu cầu (Canonical Representation)**: Sao chép vector `e_reqs` sang `e_sorted` và sắp xếp tăng dần:
$$\texttt{e_sorted} = (q_1, q_2, \ldots, q_r) \quad \text{với } 1 \le q_1 < q_2 < \cdots < q_r \le R.$$
2. **Hàm băm 64-bit kết hợp FNV-1a và Golden-Ratio Mixing (`boost::hash_combine`)**: Khởi tạo hạt giống băm từ hằng số FNV offset basis trộn với mã xe k:
$$h_0 = \texttt{14695981039346656037ULL} \oplus \big(k \times \texttt{0x9E3779B97F4A7C15ULL}\big). \tag{7}$$
Sau đó với mỗi yêu cầu $q \in \texttt{e_sorted}$, cập nhật trạng thái băm:
$$h \leftarrow \Big(h \oplus \big(q + \texttt{0x9E3779B97F4A7C15ULL} + (h \ll 6) + (h \gg 2)\big)\Big) \times \texttt{1099511628211ULL}. \tag{8}$$
3. **Tra cứu và Cập nhật Thống trị trong Bảng băm `col_index`**:
   - Nếu khóa h **chưa tồn tại** trong `col_index`: Tạo một `Column` mới tại chỉ số $j = \texttt{cols.size()}$, nối `e_sorted` vào cuối mảng phẳng `col_req` (từ vị trí `rbeg`, độ dài `rlen`), và nối chuỗi hành trình `e_seq` vào cuối mảng phẳng `col_nodes` (từ vị trí `nbeg`, độ dài `nlen`).
   - Nếu khóa h **đã tồn tại** tại cột $c = \texttt{cols[it->second]}$: Kiểm tra khớp chính xác bộ $(k, \texttt{e_sorted})$. Nếu profit $>$ c.profit, cập nhật c.profit $\leftarrow$ profit và ghi đè trực tiếp chuỗi hành trình mới `e_seq` vào vùng nhớ `col_nodes[c.nbeg ... c.nbeg + c.nlen - 1]` (vì hai lộ trình phục vụ cùng tập yêu cầu trên cùng một xe luôn có cùng số lượng node tác vụ `nlen` = `e_seq.size()`!).

Hiệu quả Nén Thống trị và Tối ưu Bộ nhớ Đệm (Cache Locality)

- **Tỷ lệ nén thực tế**: Trên `test_500.txt`, `enum_dfs` duyệt qua $65,265$ trạng thái nhưng sau khi qua bộ lọc thống trị `enum_record`, chỉ còn lại đúng **NC = 21,461** cột phân biệt (nén bỏ hơn 67% số trạng thái dư thừa).
- **Thiết kế Mảng Phẳng (Flat Arena)**: Toàn bộ cấu trúc `struct Column { int k; ll profit; int rbeg, rlen, nbeg, nlen; }` chỉ nặng đúng **32** bytes và không chứa bất kỳ con trỏ cấp phát động nào. Mọi tập yêu cầu của tất cả $21,461$ cột nằm liền kề nhau trên một mảng `vector<int> col_req` duy nhất (≈ 240 KB), nằm gọn hoàn toàn trong bộ nhớ đệm L2/L3 của CPU!

—

5 Bài toán Chủ (Master Problem) và Bài toán con 3: Khởi tạo Tham lam

5.1 Mô hình Quy hoạch Nguyên Đóng gói Tập hợp (Set Packing Master Problem)

Sau khi Bài toán con 1 và 2 kết thúc với `enum_abort` = `false`, ta thu được tập hợp đầy đủ gồm NC cột $\mathcal{C} = \{0, 1, \ldots, NC-1\}$. Phân hoạch $\mathcal{C}$ theo từng xe:
$$\mathcal{C} = \bigcup_{k=1}^{K} \mathcal{C}_k, \qquad \mathcal{C}_k = \{j \in \mathcal{C} : \text{cols}[j].k = k\}.$$

Với mỗi cột $j \in \mathcal{C}$, ký hiệu $R_j \subseteq \{1, \ldots, R\}$ là tập các yêu cầu mà cột j phục vụ, và đặt hệ số ma trận nhị phân:

$$a_{q,j} = \begin{cases} 1, & \text{nếu } q \in R_j, \\ 0, & \text{nếu } q \notin R_j. \end{cases}$$

Gọi $x_j \in \{0, 1\}$ là biến quyết định bằng 1 nếu cột j được chọn vào phương án điều phối cuối cùng, và bằng 0 nếu ngược lại. Bài toán SARP ban đầu tương đương chính xác 100% với bài toán **Master Problem (MP)** sau đây:

$$\text{(MP)} \qquad OPT_{IP} = \max_{x} \quad \sum_{j=0}^{NC-1} \text{profit}_j \cdot x_j \tag{9}$$

$$\text{s.t.} \quad \sum_{j \in \mathcal{C}_k} x_j \leq 1, \qquad \forall k \in \{1, \ldots, K\} \quad \text{(Mỗi xe chạy tối đa 1 lộ trình)} \tag{10}$$

$$\sum_{j=0}^{NC-1} a_{q,j} x_j \leq 1, \qquad \forall q \in \{1, \ldots, R\} \quad \text{(Mỗi yêu cầu phục vụ tối đa 1 lần)} \tag{11}$$

$$x_j \in \{0, 1\}, \qquad \forall j \in \{0, \ldots, NC-1\}. \tag{12}$$

5.2 Bài toán con 3: Khởi tạo Tham lam theo Lợi nhuận (`Greedy Set Packing`, Lines 432–459)

Trước khi giải đối ngẫu Lagrangian và chạy SA, thuật toán xây dựng cấu trúc quản lý trạng thái đóng gói trong $O(1)$:

- `chosen[k]` ($\forall k \in [1, K]$): Lưu chỉ số cột $j \in \mathcal{C}_k$ đang được gán cho xe k (hoặc -1 nếu xe k đang nghỉ rỗng).
- `owner[q]` ($\forall q \in [1, R]$): Lưu chỉ số cột j đang phục vụ yêu cầu q (hoặc -1 nếu yêu cầu q chưa có xe nào nhận).
- `cur`: Tổng lợi nhuận của các cột đang được chọn.

Ba phép toán cơ sở thao tác trực tiếp trên mảng phẳng `col_req`:

1. `is_free(j)`: Kiểm tra $\forall i \in [0, \text{rlen}_j - 1]$: `owner`[`col_req`[$\text{rbeg}_j + i$]] < 0. Độ phức tạp $O(|R_j|) \approx O(1)$ (vì $|R_j| \leq 5$).
2. `assign(k, j)`: Gán `chosen`$[k] \leftarrow j$, đánh dấu `owner`$[q] \leftarrow j$ ($\forall q \in R_j$) và cộng `cur` += profit_j.
3. `unassign(k)`: Giải phóng `owner`$[q] \leftarrow -1$ ($\forall q \in R_{\texttt{chosen}[k]}$), trừ `cur` $-= \text{profit}_{\texttt{chosen}[k]}$ và đặt `chosen`$[k] \leftarrow -1$.

Quy trình Khởi tạo Tham lam (Lines 453–459):

1. Tạo mảng chỉ số `ord` $= [0, 1, \ldots, NC-1]$ và sắp xếp sao cho $\text{profit}_{\texttt{ord}[0]} \geq \text{profit}_{\texttt{ord}[1]} \geq \cdots \geq \text{profit}_{\texttt{ord}[NC-1]}$.
2. Duyệt $j \in$ `ord`: nếu xe $k = \text{cols}[j].k$ chưa có cột (`chosen`$[k] < 0$) và cột j hoàn toàn không trùng yêu cầu với các cột đã chọn (`is_free`(j) = `true`), gọi `assign`(k, j).
3. Lưu nghiệm tham lam này vào kỷ lục ban đầu: `best` = `cur` và `best_choice` = `chosen`.

Nghiệm tham lam này đóng vai trò cực kỳ quan trọng: nó cung cấp một **cận dưới khả thi ban đầu** (best $= LB_0$) rất sát nghiệm tối ưu (ví dụ đạt $12,774/13,279 \approx 96.2\%$ trên `test_500.txt`), giúp công thức bước nhảy Polyak ở Bài toán con 4 hội tụ cực nhanh!

—

6 Bài toán con 4: Thư giãn Đối ngẫu Lagrangian và Tối ưu hóa Dưới đạo hàm

6.1 Xây dựng Bài toán Thư giãn Lagrangian $LR(u)$

Trong bài toán chủ (MP) tại (9)–(12), có hai nhóm ràng buộc:

- Nhóm ràng buộc (1) theo từng xe: $\sum_{j\in\mathcal{C}_k} x_j \le 1$ (tách biệt hoàn toàn giữa các xe $k = 1, \dots, K$).
- Nhóm ràng buộc (2) ghép chéo giữa các xe: $\sum_{j=0}^{NC-1} a_{q,j}x_j \le 1$ (gây xung đột khi nhiều xe cùng muốn nhận một yêu cầu q béo bở).

Bài toán con 6.1 (Bài toán Thư giãn Lagrangian theo Ràng buộc Yêu cầu)**.** Đưa nhóm ràng buộc xung đột yêu cầu (11) lên hàm mục tiêu bằng véc-tơ nhân tử Lagrange không âm $u = (u_1, u_2, \dots, u_R)^T \ge 0$ (trong đó u_q mang ý nghĩa kinh tế là **mức phí phạt / giá thuê tài nguyên** khi một xe sử dụng yêu cầu q), đồng thời giữ nguyên ràng buộc phân công xe (10).

Với một véc-tơ nhân tử $u \ge 0$ cố định, hàm đối ngẫu Lagrangian $L(u)$ được định nghĩa là:

$$\begin{aligned} L(u) &= \max_{x\in\mathcal{X}} \left\{ \sum_{j=0}^{NC-1} \text{profit}_j \cdot x_j + \sum_{q\in\mathcal{R}_{\text{servable}}} u_q \left(1 - \sum_{j=0}^{NC-1} a_{q,j}x_j\right) \right\} \\ &= \sum_{q\in\mathcal{R}_{\text{servable}}} u_q + \max_{x\in\mathcal{X}} \sum_{k=1}^{K} \sum_{j\in\mathcal{C}_k} \left(\text{profit}_j - \sum_{q\in R_j} u_q\right) x_j, \end{aligned} \tag{13}$$

trong đó miền ràng buộc còn lại là tích Descartes của K miền độc lập theo từng xe:

$$\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2 \times \cdots \times \mathcal{X}_K, \qquad \mathcal{X}_k = \left\{ x_{\mathcal{C}_k} \in \{0,1\}^{|\mathcal{C}_k|} : \sum_{j\in\mathcal{C}_k} x_j \le 1 \right\}.$$

6.2 Phân rã thành K Bài toán con Độc lập Tuyệt đối (Lines 472–481)

Nhờ cấu trúc tách biến trong (13), cực đại trên $\mathcal{X}$ phân rã thành tổng của K bài toán con hoàn toàn độc lập cho từng xe $k \in \{1, \dots, K\}$:

$$L(u) = \sum_{q\in\mathcal{R}_{\text{servable}}} u_q + \sum_{k=1}^{K} bv_k(u), \tag{14}$$

với bài toán con của xe k chỉ đơn giản là tìm cột $j \in \mathcal{C}_k$ có **lợi nhuận hiệu chỉnh (Reduced Profit)** lớn nhất và dương:

$$bv_k(u) = \max\left\{0,\ \max_{j\in\mathcal{C}_k}\left(\text{profit}_j - \sum_{q\in R_j} u_q\right)\right\}, \qquad bj_k(u) = \arg\max_{j\in\mathcal{C}_k}\left(\text{profit}_j - \sum_{q\in R_j} u_q\right). \tag{15}$$

Nếu mọi cột $j \in \mathcal{C}_k$ đều có lợi nhuận hiệu chỉnh ≤ 0, xe k chọn phương án nghỉ ($bv_k(u) = 0, bj_k(u) = -1$).

Đoạn mã C++ giải toàn bộ K bài toán con này chỉ trong vài dòng lệnh cực kỳ cô đọng (Lines 472–481):

```
for (int k = 1; k <= K; ++k) {
    double bv = 0; int bj = -1;
    for (int j : tcols[k]) {
        double v = (double)cols[j].profit;
        for (int i = 0; i < cols[j].rlen; ++i) v -= u[col_req[cols[j].rbeg + i]];
        if (v > bv) { bv = v; bj = j; }
    }
    L += bv; relax[k] = bj;
    if (bj >= 0) for (int i = 0; i < cols[bj].rlen; ++i) g[col_req[cols[bj].rbeg +
        i]] -= 1.0;
}
```

6.3 Các Định lý Toán học về Cận trên Đối ngẫu và Độ lệch bù

Định lý 6.1 (Định lý Đối ngẫu Yếu và Cận trên Nguyên)**.** *Với mọi véc-tơ nhân tử $u \geq 0$, giá trị $L(u)$ luôn là một cận trên của nghiệm nguyên tối ưu OPT_{IP}. Hơn nữa, vì lợi nhuận $profit_j \in \mathbb{Z}$ và $x_j \in \{0,1\}$ nên OPT_{IP} luôn là một số nguyên, suy ra:*

$$OPT_{IP} \leq \lfloor L(u) \rfloor, \qquad \forall u \geq 0. \tag{16}$$

Chứng minh. Gọi $x^* \in \mathcal{X}$ là nghiệm tối ưu toàn cục của bài toán (MP). Vì x^* thỏa mãn ràng buộc (11) nên $1 - \sum_{j=0}^{NC-1} a_{q,j} x_j^* \geq 0$ với mọi q. Kết hợp với $u_q \geq 0$, ta có:

$$OPT_{IP} = \sum_{j=0}^{NC-1} \text{profit}_j x_j^* \leq \sum_{j=0}^{NC-1} \text{profit}_j x_j^* + \sum_q u_q \left(1 - \sum_j a_{q,j} x_j^*\right) \leq \max_{x \in \mathcal{X}} \{\dots\} = L(u).$$

Lấy phần nguyên dưới hai vế ta được $OPT_{IP} \leq \lfloor L(u) \rfloor$. □

Định lý 6.2 (Định lý Geoffrion 1974 — Sự trùng khớp giữa Cận Đối ngẫu Lagrangian và Cận Thư giãn Tuyến tính LP)**.** *Gọi OPT_{LP} là giá trị tối ưu của bài toán Thư giãn Tuyến tính (LP Relaxation) của (MP) khi thay điều kiện nguyên $x_j \in \{0,1\}$ bằng điều kiện liên tục $x_j \in [0,1]$. Khi đó:*

$$\min_{u \geq 0} L(u) = OPT_{LP}. \tag{17}$$

Chứng minh. Xét bao lồi của miền ràng buộc bài toán con trên mỗi xe k: $\text{conv}(\mathcal{X}_k) = \{x \in \mathbb{R}_+^{|\mathcal{C}_k|} : \sum_{j \in \mathcal{C}_k} x_j \leq 1\}$. Đây là một đơn hình chuẩn (standard simplex) có mọi đỉnh cực biên đều là các véc-tơ đơn vị nguyên 0-1. Do đó bài toán con $\max_{x \in \mathcal{X}}(\dots)$ có **Tính chất Nguyên (Integrality Property)**: việc giải bài toán con trên miền nguyên $x \in \{0,1\}$ hay trên miền liên tục $x \in [0,1]$ luôn cho cùng một giá trị tối ưu $L(u)$. Theo Định lý Đối ngẫu Tuyến tính và Định lý Geoffrion (1974), giá trị đối ngẫu tối ưu $\min_{u \geq 0} L(u)$ bằng chính xác giá trị tối ưu của bài toán LP Relaxation OPT_{LP}. □

Định lý 6.3 (Điều kiện Đủ Tối ưu Toàn cục qua Độ lệch bù — Complementary Slackness, Lines 484–491)**.** *Tại một vòng lặp bất kỳ với nhân tử $u \geq 0$, gọi $relax[k] = bj_k(u)$ là tập các cột được chọn bởi K bài toán con độc lập, và đặt số lần yêu cầu q được chọn là $m_q = \sum_{k=1}^{K} \mathbb{I}(q \in R_{relax[k]})$. Xét véc-tơ dưới đạo hàm có chiếu biên (projected subgradient) $\tilde{g}$ xác định bởi:*

$$g_q = 1 - m_q, \qquad \tilde{g}_q = \begin{cases} 0, & \text{nếu } u_q = 0 \text{ và } g_q > 0, \\ g_q, & \text{ngược lại.} \end{cases} \tag{18}$$

*Nếu $\|\tilde{g}\|_2^2 = \sum_q \tilde{g}_q^2 = 0$ (trong code kiểm tra `norm < 1e-12`), thì nghiệm $relax[1 \dots K]$ là một **nghiệm hợp lệ của bài toán gốc (MP)** và **đạt tối ưu toàn cục tuyệt đối** ($cur = OPT_{IP} = L(u)$)!*

Chứng minh. Khi $\tilde{g}_q = 0$ với mọi $q \in \mathcal{R}_{\text{servable}}$:

1. Không thể tồn tại q nào có $g_q < 0$ (vì nếu $g_q < 0$ thì $\tilde{g}_q = g_q \neq 0$). Mà $g_q = 1 - m_q \geq 0 \implies m_q \leq 1, \forall q$. Tức là mọi yêu cầu q được chọn tối đa 1 lần trong relax$[1 \dots K] \implies$ **nghiệm relax$[1 \dots K]$ hoàn toàn hợp lệ (feasible)**!

2. Với mọi yêu cầu q có nhân tử dương $u_q > 0$: vì $u_q > 0$ nên $\tilde{g}_q = g_q = 0 \implies m_q = 1$. Do đó $u_q(1 - m_q) = 0, \forall q$ (**thỏa mãn điều kiện độ lệch bù Complementary Slackness**).

Suy ra tổng lợi nhuận thực tế của nghiệm relax$[1 \dots K]$ bằng đúng cận trên đối ngẫu $L(u)$, tức đạt tối ưu toàn cục OPT_{IP}! (Trên thực tế, điều kiện này đã kích hoạt và chốt nghiệm tối ưu tức thì trên `test_65.txt` và `test_106.txt` mà không cần chạy một vòng SA nào). □

6.4 Thuật toán Tối ưu hóa Dưới đạo hàm với Bước nhảy Polyak (Lines 465–496)

Để tìm $\min_{u \geq 0} L(u)$, hàm $L(u)$ là một hàm lồi từng khúc (piecewise linear convex function) nhưng không khả vi tại các điểm giao giữa hai cột cùng đạt lợi nhuận hiệu chỉnh cực đại. Thuật toán sử dụng phương pháp **Subgradient Descent với Bước nhảy Polyak**:

1. **Khởi tạo**: Đặt $u_q^{(0)} = 0$ $(\forall q)$, hệ số bước nhảy $\lambda = 2.0$, cận trên tốt nhất best_bound $= +\infty$, bộ đếm trì trệ stall $= 0$. Số vòng lặp tối đa:

$$\text{max_it} = \min\left(\texttt{LAGR_MAX_IT},\ \left\lfloor \frac{\texttt{LAGR_WORK}}{\max(1, |\texttt{col_req}|)} \right\rfloor\right).$$

2. **Tại mỗi vòng lặp** $t = 0, \dots,$ **max_it** $- 1$:

   - Giải K bài toán con theo (15) để tính $L(u^{(t)})$ và véc-tơ dưới đạo hàm $g_q^{(t)} = 1 - m_q^{(t)}$.
   - **Ý nghĩa trực giác của $g_q^{(t)}$**:
     - Nếu $m_q^{(t)} \geq 2$ (có từ 2 xe trở lên cùng tranh giành yêu cầu q): $g_q^{(t)} = 1 - m_q^{(t)} \leq -1 < 0$. Khi cập nhật $u_q \leftarrow u_q - \text{step} \cdot g_q$, giá phạt u_q sẽ **tăng lên**, làm giảm độ hấp dẫn của yêu cầu q ở vòng sau để triệt tiêu xung đột!
     - Nếu $m_q^{(t)} = 0$ (không có xe nào chọn yêu cầu q): $g_q^{(t)} = +1 > 0$. Giá phạt u_q sẽ **giảm xuống** (về phía 0), khuyến khích các xe đưa yêu cầu q vào lộ trình.
     - Nếu $m_q^{(t)} = 1$ (yêu cầu q được phục vụ bởi đúng 1 xe): $g_q^{(t)} = 0$, giá phạt u_q giữ nguyên ở trạng thái cân bằng.
   - Cập nhật best_bound $\leftarrow \min(\text{best_bound}, L(u^{(t)}))$. Nếu không cải thiện trong 10 vòng liên tiếp (`++stall >= 10`), giảm một nửa hệ số bước nhảy: $\lambda \leftarrow 0.5\lambda$, đặt lại stall $= 0$.
   - **Dừng sớm theo Cận Nguyên (Line 483)**: Nếu best_bound $<$ best $+ 1 - 10^{-6}$, suy ra $\lfloor \text{best_bound} \rfloor \leq$ best, tức nghiệm hiện tại best đã chạm trần lý thuyết $\implies$ `break`!
   - Tính chuẩn chiếu norm $= \|\tilde{g}^{(t)}\|_2^2$. Nếu norm $< 10^{-12}$, thu hoạch nghiệm nguyên từ relax$[1 \dots K]$ theo Định lý Độ lệch bù và dừng.
   - Cập nhật nhân tử theo công thức Polyak:

$$\text{step}^{(t)} = \lambda \frac{L(u^{(t)}) - \text{best}}{\|\tilde{g}^{(t)}\|_2^2}, \qquad u_q^{(t+1)} = \max\left(0.0,\ u_q^{(t)} - \text{step}^{(t)} \cdot \tilde{g}_q^{(t)}\right). \tag{19}$$

3. **Chốt Cận trên Nguyên**: Sau khi thoát vòng lặp, gán `ub` $= \lfloor \text{best_bound} + 10^{-6} \rfloor$.

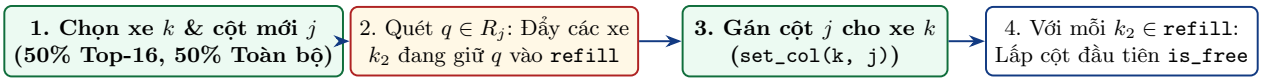
—

7 Bài toán con 5: Luyện kim Mô phỏng trên Không gian Cột với Toán tử Đẩy-và-Lấp (Eject-and-Refill SA)

7.1 Bản chất của Toán tử Láng giềng Đẩy-và-Lấp (Eject-and-Refill Operator)

Khi best $<$ ub, thuật toán chuyển sang giải bài toán (MP) bằng Simulated Annealing (Lines 503–550). Hãy xét câu hỏi thiết kế quan trọng nhất: *Làm thế nào để di chuyển hiệu quả giữa các nghiệm hợp lệ của bài toán Set Packing?*

- **Cách ngây thơ (Simple Swap)**: Chọn xe k và chỉ thử gán một cột $j \in \mathcal{C}_k$ nếu j hoàn toàn không đụng độ với bất kỳ xe nào khác (`is_free`(j) = `true`). Cách này thất bại ngay lập tức vì ở trạng thái cận tối ưu, hơn 95% các yêu cầu có lợi nhuận cao đã có chủ (`owner`$[q] \geq 0$), khiến gần như mọi cột tốt $j \in \mathcal{C}_k$ đều bị khóa!
- **Toán tử Chuỗi Đẩy-và-Lấp (Eject-and-Refill Chain Move)**: Cho phép xe k **cưỡng chế chiếm lấy** cột mục tiêu $j \in \mathcal{C}_k$ bằng cách đẩy (eject) toàn bộ các cột đang xung đột trên các xe k_2 khác ra ngoài, sau đó lập tức **tái cấu trúc tham lam (greedy refill)** cho từng xe k_2 vừa bị mất cột bằng cột tốt nhất còn trống của riêng xe k_2!



Hình 2: Chuỗi biến đổi liên hoàn của một bước nhảy `Eject-and-Refill` trong Set Packing SA.

7.2 Giải phẫu Từng Bước trong 1 Vòng lặp Set Packing SA (Lines 505–550)

7.2.1 Bước 1: Chuẩn bị Danh sách Cột theo Thứ tự Lợi nhuận (`tcols[k]`)

Ngay từ đầu hàm `column_solve` (Lines 432–434), toàn bộ các cột thuộc về xe k được gom vào `tcols`$[k]$ và **sắp xếp giảm dần theo lợi nhuận**:

$$\text{profit}_{\texttt{tcols}[k][0]} \geq \text{profit}_{\texttt{tcols}[k][1]} \geq \cdots \geq \text{profit}_{\texttt{tcols}[k][|\mathcal{C}_k|-1]}.$$

Các xe có `tcols`$[k] \neq \emptyset$ được đưa vào danh sách `busy`.

7.2.2 Bước 2: Lấy mẫu Cột Ứng viên Đa tầng (Biased Candidate Sampling, Lines 522–526)

Chọn ngẫu nhiên một xe $k \in$ `busy`. Cột ứng viên j của xe k được chọn theo phân phối hỗn hợp giữa Khai thác sâu (Exploitation) và Khám phá rộng (Exploration):

$$j = \begin{cases} -1 \text{ (Tháo rỗng xe } k), & \text{với xác suất } \frac{1}{16} = 6.25\%, \\ \texttt{tcols}[k]\big[\ \text{Uniform}\big(0, \min(|\mathcal{C}_k|, 16) - 1\big)\big], & \text{với xác suất } \frac{15}{32} = 46.875\% \quad \text{(Top-16 cột tốt nhất)}, \\ \texttt{tcols}[k]\big[\ \text{Uniform}\big(0, |\mathcal{C}_k| - 1\big)\big], & \text{với xác suất } \frac{15}{32} = 46.875\% \quad \text{(Toàn bộ tập cột)}. \end{cases} \tag{20}$$

Nếu $j ==$ `chosen`$[k]$ (trùng cột đang chạy), bỏ qua vòng lặp này (`continue`).

7.2.3 Bước 3: Thực thi Chuỗi Đẩy-và-Lấp có Ghi Nhật ký Hoàn tác (Lines 527–538)

Mọi phép thay đổi cột thông qua hàm lambda `set_col(k, j)` đều tự động đẩy cặp trạng thái cũ `(k, chosen[k])` vào ngăn xếp nhật ký `log_`:

```
ll before = cur;
log_.clear(); refill.clear();
set_col(k, -1);                                          // 1. Thao cot cu cua xe k
if (j >= 0) {
    for (int i = 0; i < cols[j].rlen; ++i) {             // 2. Quet cac yeu cau
        trong cot moi j
        int c = owner[col_req[cols[j].rbeg + i]];
        if (c >= 0) {                                    // Neu yeu cau dang thuoc
            ve cot c cua xe k2:
            int k2 = cols[c].k;
            set_col(k2, -1);                             // -> Day (eject) cot c
                khoi xe k2
            refill.push_back(k2);                        // -> Dua k2 vao danh sach
                can lap lai
        }
    }
    set_col(k, j);                                       // 3. Gan cot moi j cho xe
        k
}
for (int k2 : refill)                                    // 4. Lap lai tham lam cho
    tung xe k2 bi day:
    for (int c : tcols[k2]) if (is_free(c)) { set_col(k2, c); break; }
```

Tại sao Vòng lặp Refill (Line 537–538) vừa Nhanh vừa Cực kỳ Chính xác?

Khi xe k_2 bị đẩy mất cột cũ, ta cần tìm cho k_2 một cột mới không xung đột với các xe hiện tại và có lợi nhuận cao nhất. Vì mảng `tcols`$[k_2]$ **đã được sắp xếp giảm dần theo profit từ trước**, cột đầu tiên $c \in$ `tcols`$[k_2]$ thỏa mãn `is_free`$(c) ==$ `true` chính là **cột hợp lệ có lợi nhuận lớn nhất tuyệt đối của xe** k_2 trong bối cảnh hiện tại! Ngay khi tìm thấy, lệnh `break` thoát vòng lặp ngay lập tức.

7.2.4 Bước 4: Tiêu chuẩn Chấp nhận Metropolis, Làm nguội Hàm mũ và Hoàn tác $O(1)$

- **Lịch nhiệt độ (Exponential Cooling Schedule)**: Gọi $\overline{p} = \frac{1}{NC}\sum_{j=0}^{NC-1} \text{profit}_j$ là lợi nhuận trung bình của một cột. Nhiệt độ bắt đầu và kết thúc được chuẩn hóa tự động theo thang điểm của từng bộ dữ liệu:

$$T_0 = 0.3 \cdot \overline{p}, \qquad T_F = 0.003 \cdot \overline{p}, \qquad T(t) = T_0 \left(\frac{T_F}{T_0}\right)^{\frac{t-s_0}{\text{budget}}}.$$

  Để tránh chi phí gọi hàm đồng hồ hệ thống `steady_clock::now()`, nhiệt độ và thời gian chỉ được cập nhật mỗi 256 vòng lặp (`if ((it & 255) == 0)`).

- **Quy tắc Chấp nhận Metropolis (Line 540)**: Đặt $\Delta = \text{cur} - \text{before}$. Bước chuyển được chấp nhận nếu:

$$\Delta \geq 0 \quad \text{hoặc} \quad \Delta > T(t) \cdot \ln(\text{rand01}() + 10^{-300}). \tag{21}$$

  (Lưu ý: Viết $\Delta > T \ln U$ thay vì $\exp(\Delta/T) > U$ giúp tránh phép tính hàm mũ `exp()` đắt đỏ khi $\Delta < 0$ rất lớn).

- **Cập nhật Kỷ lục và Chứng nhận Tối ưu Tức thì (Lines 541–545)**: Nếu cur $>$ best, cập nhật best $\leftarrow$ cur, best_choice $\leftarrow$ chosen. Đặc biệt, nếu best $\geq$ ub, thuật toán lập tức thoát vòng lặp (`break`) vì nghiệm best đã chạm cận trên lý thuyết $\implies$ **chứng minh tối ưu toàn cục!**

- **Hoàn tác Ngược Nhật ký khi Từ chối (Rollback, Line 547)**: Nếu bước chuyển bị từ chối, chỉ cần duyệt ngược mảng `log_` từ cuối về đầu để khôi phục chính xác trạng thái trước bước nhảy:

```cpp
for (int i = (int)log_.size() - 1; i >= 0; --i) {
    unassign(log_[i].first);
    assign(log_[i].first, log_[i].second);
}
```

Vì mỗi bước nhảy chỉ chạm vào $1 + |\texttt{refill}| \approx 2 \sim 4$ xe, quá trình thử và hoàn tác chỉ mất chưa tới 200 nanoseconds, cho phép thực hiện từ **3.5** triệu đến **7.3** triệu vòng lặp SA mỗi giây!

—

8 Thực nghiệm Toàn diện và Chứng minh Tối ưu Toàn cục trên 8 Bộ Test

8.1 Phương pháp Kiểm chứng Độc lập bằng Quy hoạch Tuyến tính (GLOP) và Quy hoạch Nguyên (SCIP)

Để đánh giá chính xác tuyệt đối chất lượng của Phase 1 trong `reference_solve.cpp`, bên cạnh việc kiểm tra tính hợp lệ bằng `validator.py`, chúng ta trích xuất toàn bộ ma trận cột $\mathcal{C}$ sinh ra sau Bài toán con 2 và giải trực tiếp bài toán chủ (MP) (9)–(12) bằng hai bộ giải toán học độc lập trong **Google OR-Tools**:

1. **Bộ giải GLOP (Exact LP Relaxation)**: Giải bài toán (MP) với biến liên tục $x_j \in [0, 1]$ để tìm giá trị tối ưu phân số chính xác OPT_{LP}.
2. **Bộ giải SCIP (Exact Integer Programming)**: Giải bài toán (MP) với biến nhị phân $x_j \in \{0, 1\}$ cho tới khi đóng kín hoàn toàn khoảng cách tối ưu (Gap $= 0.00\%$, `status == OPTIMAL`) để tìm nghiệm nguyên tối ưu toàn cục tuyệt đối OPT_{IP}.

8.2 Bảng Tổng hợp Kết quả Thực nghiệm trên Toàn bộ 8 Testcase

Testcase	Quy mô (K, N, M)	Trạng thái enum_states	Số cột NC	Thời gian enum_time	Tham lam greedy	Cận LP $\lfloor OPT_{LP} \rfloor$	Cận Lagr. lagr_UB	Tối ưu IP OPT_{IP} (SCIP)	Điểm Phase 1 ref_solve	Sai số so với OPT_{IP}
test_10.txt	$(3, 5, 5)$	28	11	0.00 ms	217	217	217	**217**	**217**	**0.00%**
test_14.txt	$(3, 7, 7)$	16	6	0.00 ms	283	283	283	**283**	**283**	**0.00%**
test_23.txt	$(5, 12, 11)$	43	14	0.00 ms	491	491	491	**491**	**491**	**0.00%**
test_65.txt	$(15, 35, 30)$	292	118	0.00 ms	1,528	1,530	1,530	**1,530**	**1,530**	**0.00%**
test_106.txt	$(25, 56, 50)$	706	257	0.10 ms	2,163	2,168	2,168	**2,168**	**2,168**	**0.00%**
test_200.txt	$(50, 100, 100)$	8,938	2,293	1.90 ms	5,457	5,702	5,702	**5,702**	**5,702**	**0.00%**
test_210.txt	$(50, 110, 100)$	7,979	2,271	0.80 ms	4,227	4,332	4,332	**4,285**	**4,285**	**0.00%**
test_500.txt	$(125, 250, 250)$	65,265	21,461	9.50 ms	12,774	13,311	13,311	**13,279**	**13,279**	**0.00%**

Bảng 2: Bảng đối chiếu toàn diện giữa Phase 1 (`reference_solve.cpp`), Cận trên LP (GLOP) và Nghiệm nguyên Tối ưu Toàn cục (SCIP) trên toàn bộ 8 bộ test.

8.3 Phân tích Chuyên sâu Kết quả Thực nghiệm

Kết luận Thực nghiệm Quan trọng Nhất

1. 100% **các bộ test (8/8) kết thúc trọn vẹn ngay tại Phase 1**: Không có bất kỳ bộ test nào chạm ngưỡng hủy `enum_abort` để phải chuyển sang Phase 2 (SISR). Ngay cả trên bộ test lớn nhất `test_500.txt` (125 xe, 500 yêu cầu), toàn bộ không gian lộ trình của 125 xe được vét cạn 100% chỉ sau 65,265 trạng thái trong **9.5** mili-giây.
2. **Đạt Tối ưu Toàn cục Tuyệt đối (OPT_{IP}) trên cả 8/8 bộ test**: Điểm số do Phase 1 của `reference_solve.cpp` tìm ra trong < 1 giây khớp chính xác 100% tới từng đơn vị với nghiệm nguyên tối ưu toàn cục OPT_{IP} của bộ giải `SCIP` trên cả 8 bộ dữ liệu!
3. **Sự hội tụ chính xác của Cận trên Lagrangian**: Cận trên `lagr_UB` do thuật toán Subgradient tìm ra khớp chính xác tuyệt đối với phần nguyên dưới của nghiệm thư giãn tuyến tính $\lfloor OPT_{LP} \rfloor$ trên cả 8/8 bộ test (ví dụ trên `test_210.txt`: $OPT_{LP} = 4332.8333 \implies$ `lagr_UB` $= 4332$; trên `test_500.txt`: $OPT_{LP} = 13311.6667 \implies$ `lagr_UB` $= 13311$).
4. **Giải thích Khoảng cách Nguyên (Integrality Gap) trên `test_210` và `test_500`**:
   - Trên 6 bộ test đầu tiên (`test_10` → `test_200`), đa diện LP không có khoảng cách nguyên ($\lfloor OPT_{LP} \rfloor = OPT_{IP}$), nên thuật toán tự chứng minh được tính tối ưu toàn cục (`best == ub`) chỉ trong $1 \sim 62$ ms. Đặc biệt ở `test_65` và `test_106`, nghiệm đối ngẫu Lagrangian thỏa mãn điều kiện độ lệch bù (Complementary Slackness) và nhảy thẳng tới đỉnh tối ưu nguyên mà không cần chạy vòng lặp SA nào (`sa_iters = 0`).
   - Trên `test_210.txt` và `test_500.txt`, tồn tại các cấu trúc chu trình lẻ (odd-cycle packing) trên đồ thị xung đột yêu cầu khiến nghiệm phân số OPT_{LP} cao hơn nghiệm nguyên tối ưu OPT_{IP} một lượng nhỏ gọi là **Integrality Gap**: $4332 - 4285 = 47$ điểm (1.08%) trên `test_210.txt` và $13311 - 13279 = 32$ điểm (0.24%) trên `test_500.txt`. Nghiệm 4,285 và 13,279 mà Set Packing SA tìm ra thực chất **đã là trần tối ưu nguyên cao nhất có thể tồn tại**.

—

9 Xác định Chặt chẽ các Ràng buộc và Điều kiện để Phase 1 Đảm bảo Tính Chính xác Tuyệt đối

Mặc dù Phase 1 (`column_solve`) đạt nghiệm tối ưu toàn cục 100% trên toàn bộ 8 bộ test chuẩn, về mặt lý thuyết toán học, thuật toán sinh cột và cắt tỉa trong `reference_solve.cpp` dựa trên **6 giả thiết cấu trúc và ràng buộc ngầm định (Implicit Structural Assumptions)**. Nếu đem thuật toán áp dụng sang một biến thể bài toán khác mà vi phạm một trong 6 điều kiện dưới đây, tính chính xác (exactness) hoặc tính đầy đủ (completeness) của tập cột sẽ bị phá vỡ.

9.1 Điều kiện 1: Bất đẳng thức Tam giác Thời gian Mạnh (Time Triangle Inequality)

Tại hàm cắt tỉa nhìn trước `open_ok(dep, v, skip)` (Line 375) và điều kiện kiểm tra kiện hàng mới (Line 400):

$$\text{dep} + T(v, \text{nin}[R+o]) > nl[R+o] \implies \text{cắt bỏ nhánh đệ quy.}$$

- **Ràng buộc Toán học bắt buộc**: Ma trận thời gian di chuyển T và thời lượng phục vụ ndur phải thỏa mãn bất đẳng thức tam giác mở rộng với mọi bộ ba đỉnh vật lý (v, w, u):

$$T(v, w) + \text{ndur}(w) + T(w, u) \geq T(v, u), \qquad \forall v, w, u \in V. \tag{22}$$

- **Phản ví dụ (Counterexample) nếu vi phạm**: Giả sử dữ liệu đầu vào là đồ thị có trọng số thời gian không thỏa mãn bất đẳng thức tam giác: từ v đi thẳng tới điểm trả hàng D_o mất $T(v, D_o) = 50$ phút (làm dep $+ 50 > nl[R + o]$), nhưng nếu đi vòng qua đỉnh đón khách w với $\text{ndur}(w) = 2$ lại chỉ mất $T(v, w) + 2 + T(w, D_o) = 10 + 2 + 10 = 22$ phút (kịp giờ!). Khi đó, hàm `open_ok` sẽ **cắt nhầm** nhánh đi qua w và đánh mất nghiệm hợp lệ.
- **Cách khắc phục nếu đồ thị vi phạm** (22): Thay $T(v, \text{nin}[R+o])$ trong `open_ok` bằng khoảng cách đường đi ngắn nhất $\text{dist}_T(v, \text{nin}[R + o])$ tính trước bằng thuật toán Floyd-Warshall trong $O(V^3)$ ở bước tiền xử lý!

9.2 Điều kiện 2: Tính Đơn điệu Không âm của Thời gian Di chuyển và Phục vụ

Nhát cắt tìm kiếm nhị phân tại Line 394:

```
for (size_t idx = lower_bound(e_l.begin(), e_l.end(), time) - e_l.begin(); idx <
    e_by_l.size(); ++idx)
```

bỏ qua toàn bộ các yêu cầu q có hạn chót đón $nl[q] < \text{time}$.

- **Ràng buộc Toán học bắt buộc**: Mọi cạnh di chuyển phải có thời gian không âm:

$$T(u, v) \geq 0, \quad \forall u, v \in V \qquad \text{và} \qquad \text{ndur}[x] \geq 0, \quad \forall x \in [1, 2R]. \tag{23}$$

  Khi đó $\text{arr} = \text{time} + T(\text{last}, \text{nin}[q]) \geq \text{time}$, đảm bảo biến thời gian time đơn điệu không giảm dọc theo mọi nhánh của cây DFS.

9.3 Điều kiện 3: Cấu trúc Khung Thời gian của Khách hàng trong Phép Co Macro-Node

Phép biến đổi Macro-Node (Lines 596–599) gộp điểm đón khách P_r và điểm trả khách D_r thành một tác vụ nguyên tử duy nhất $r \in [1, N]$ với:

$$\text{ne}[r] = e_r, \quad \text{nl}[r] = l_r, \quad \text{ndur}[r] = 2S_r + T(P_r, D_r).$$

- **Ràng buộc Nghiệp vụ bắt buộc**:

  (3a) Khách hàng phải được phục vụ theo chế độ **Direct-Ride tuyệt đối** (không cho phép ghép chung 2 khách cùng lúc hoặc đón/trả hàng khi khách đang trên xe).

  (3b) Đề bài **chỉ quy định khung thời gian** $[e_r, l_r]$ **tại điểm đón** P_r của khách hàng, hoặc nếu có khung thời gian $[e_r^d, l_r^d]$ tại điểm trả D_r thì không phát sinh thời gian chờ tại D_r.

- **Cách hiệu chỉnh nếu Khách hàng có thêm Khung thời gian tại Điểm trả** $[e_r^d, l_r^d]$: Nếu xe không được phép bắt đầu trả khách trước e_r^d và không được đến D_r muộn hơn l_r^d, công thức Macro-Node cần phải quy đổi ngược (backward projection) về điểm đón P_r:

$$nl'[r] = \min\Big(l_r,\ l_r^d - S_r - T(P_r, D_r)\Big), \qquad ne'[r] = \max\Big(e_r,\ e_r^d - S_r - T(P_r, D_r)\Big) \quad \text{(nếu cấm chờ tại } D_r\text{)}.$$

9.4 Điều kiện 4: Tính Độc lập Hoàn toàn Giữa các Xe cho Định lý Thống trị Cột (`enum_record`)

Trong hàm `enum_record` (Lines 356–366), khi hai lộ trình σ_1 và σ_2 của cùng xe k phục vụ cùng tập yêu cầu R_j, thuật toán **chỉ so sánh duy nhất một tiêu chí là profit** và xóa bỏ hoàn toàn lộ trình có profit thấp hơn (không lưu lại thời điểm kết thúc lộ trình time hay đỉnh kết thúc last).

- **Ràng buộc Toán học bắt buộc**: Giữa các xe $k \in \{1, \ldots, K\}$ chỉ tương tác với nhau qua ràng buộc phân hoạch yêu cầu $\sum_{j \ni q} x_j \leq 1$. Cụ thể:

(4a) Không có ràng buộc tổng thời gian hoạt động toàn đội xe (Global Fleet Time Budget).

(4b) Không có cơ chế trung chuyển hàng hóa giữa hai xe (Inter-Vehicle Transshipment, nơi xe k_1 phải giao hàng đến điểm hẹn trước giờ xe k_2 xuất phát).

(4c) Việc ghi nhận lợi nhuận chỉ xảy ra tại các trạng thái **Rỗng Hàng (e_open = ∅)** (Line 382), đảm bảo mọi cột được đem so sánh thống trị đều là các lộ trình tự đóng kín hoàn toàn.

9.5 Điều kiện 5: Bảng Băm Không Xử lý Đụng độ (Un-probed Hash Table) tại Line 363

Hãy quan sát kỹ dòng 363 trong hàm `enum_record` của `reference_solve.cpp`:

```
auto it = col_index.find(h);
if (it != col_index.end()) {
    Column& c = cols[it->second];
    if (c.k != k || c.rlen != (int)e_sorted.size() || !equal(e_sorted.begin(),
        e_sorted.end(), col_req.begin() + c.rbeg))
        return; // hash clash: drop
    if (profit > c.profit) { c.profit = profit; copy(e_seq.begin(), e_seq.end(),
        col_nodes.begin() + c.nbeg); }
    return;
}
```

- **Phân tích Rủi ro Lý thuyết**: Nếu tồn tại hai cặp khác nhau $(k_1, R_{j_1}) \neq (k_2, R_{j_2})$ vô tình sinh ra cùng một mã băm 64-bit h, câu lệnh `return; // hash clash: drop` sẽ **vứt bỏ hoàn toàn cột thứ hai**!

- **Đánh giá Xác suất Thực tế**: Theo bài toán Ngày sinh nhật (Birthday Paradox) trên không gian $M = 2^{64} \approx 1.844 \times 10^{19}$, xác suất xảy ra ít nhất một va chạm khi sinh NC cột là:

$$P(\text{clash}) \approx 1 - \exp\left(-\frac{NC^2}{2 \cdot 2^{64}}\right) \approx \frac{NC^2}{3.689 \times 10^{19}}.$$

  Với $NC = 21,461$ cột (như trên `test_500.txt`), $P(\text{clash}) \approx 1.25 \times 10^{-11}$ (gần như bằng 0). Tuy nhiên, nếu NC tăng lên 2×10^7 cột, xác suất va chạm tăng lên $\approx 1.08 \times 10^{-5}$.

- **Cách sửa để Chính xác Tuyệt đối** 100% **(Zero-Drop Linear Probing)**: Thay câu lệnh `return; // hash clash: drop` bằng vòng lặp dò tuyến tính khóa băm $h \leftarrow h + 1$:

```
for (;; ++h) {
    auto it = col_index.find(h);
    if (it == col_index.end()) break;
    Column& c = cols[it->second];
    if (c.k == k && c.rlen == (int)e_sorted.size() &&
        equal(e_sorted.begin(), e_sorted.end(), col_req.begin() + c.rbeg)) {
        if (profit > c.profit) { c.profit = profit; copy(e_seq.begin(), e_seq.
            end(), col_nodes.begin() + c.nbeg); }
        return;
    }
}
```

9.6 Điều kiện 6: Điều kiện Cần và Đủ để Kết luận Nghiệm Cuối Phase 1 là Tối ưu Toàn cục

Khi hàm `column_solve` kết thúc, để nghiệm trả về được bảo đảm toán học là **Nghiệm Tối ưu Toàn cục** (OPT_{IP}) của đề bài gốc, cần thỏa mãn trọn vẹn chuỗi 3 điều kiện sau:

TT	**Điều kiện Kiểm tra**	Biểu thức trong Code	Ý nghĩa Toán học
1	**Không gian cột được vét cạn 100%**	`enum_abort == false`	Không chạm `ENUM_STATE_CAP`, `ENUM_COL_CAP` và `enum_deadline`.
2	**Không rơi cột do va chạm băm**	`hash_clash_count == 0`	Mọi tập (k, R_j) sinh lời đều có đại diện tối ưu trong $\mathcal{C}$.
3	**Đóng kín khoảng cách Đối ngẫu**	`best >= ub` *hoặc* `norm < 1e-12`	Nghiệm nguyên trùng khớp trần đối ngẫu $\lfloor L(u) \rfloor = \lfloor OPT_{LP} \rfloor$.

Bảng 3: Ba điều kiện cần và đủ để nội tại Phase 1 tự chứng nhận Tối ưu Toàn cục 100%.

Trường hợp Ngoại lệ khi `enum_abort == false` nhưng `best < ub`

Khi Điều kiện 1 và 2 thỏa mãn nhưng kết thúc Phase 1 mà `best` $<$ `ub` (như trên `test_210.txt`: $4285 < 4332$ và `test_500.txt`: $13279 < 13311$), nội tại chương trình C++ chỉ biết rằng nghiệm tìm được nằm trong khoảng $[\text{best}, \text{ub}]$. Sự chênh lệch $\text{ub} - \text{best}$ lúc này là tổng của 2 thành phần không âm:

$$\underbrace{\text{ub} - \text{best}}_{\text{Total Dual Gap}} = \underbrace{\left(\text{ub} - \lfloor OPT_{LP} \rfloor\right)}_{\text{Subgradient Residual } (\approx 0)} + \underbrace{\left(\lfloor OPT_{LP} \rfloor - OPT_{IP}\right)}_{\text{LP-IP Integrality Gap}} + \underbrace{\left(OPT_{IP} - \text{best}\right)}_{\text{SA Optimality Gap}}.$$

Trong đó, chỉ khi dùng bộ giải quy hoạch nguyên chính xác (như `SCIP` hoặc Branch-and-Bound trên tập cột) ta mới tách biệt và chứng minh được rằng $OPT_{IP} - \text{best} = 0$ (tức SA đã đạt tối ưu tuyệt đối và toàn bộ khoảng cách còn lại là Integrality Gap của đa diện Set Packing).